

Atlas Training Migration — All Documents

None of your files are damaged. The Acrobat error just means a **.md / .csv / .py** file was opened with a PDF reader. Below is every file and how to open it; the readable docs follow in full.

File guide — what each file is

File	How to open
APPROVAL.md	Text/Markdown — open with Notepad, or read it in this PDF
Atlas_Final_Steps.pdf	PDF — opens with Acrobat / any PDF viewer
Atlas_Training_Master_VERIFIED.xlsx	Spreadsheet — open with Excel
CLAUDE_CHROME_HANDOVER.md	Text/Markdown — open with Notepad, or read it in this PDF
SAFETY_PROTOCOL.md	Text/Markdown — open with Notepad, or read it in this PDF
_index.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/AGENT_RUNBOOK.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/EXCEPTIONS.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/EXPIRY_FINDING.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/FIELD_MAPPING.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/LOGIN_FLOW.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/ORCHESTRATION.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/PORTAL_WALKTHROUGH_TEMPLATE.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/QA_VERIFICATION.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/atlas_swarm.py	Python script — open/edit with Notepad; RUN with `python`
agent-swarm/entry_log.csv	Data — open with Excel (or Notepad)
agent-swarm/portal-captures/atlas-login.png	Image — opens with Photos
agent-swarm/swarm/README.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/swarm/SELECTOR_WORKSHEET.md	Text/Markdown — open with Notepad, or read it in this PDF
agent-swarm/swarm/WINDOWS_QUICKSTART.md	Text/Markdown — open with Notepad, or read it in this PDF

File	How to open
agent-swarm/swarm/capture_selectors.js	JavaScript snippet — open with Notepad
agent-swarm/swarm/config.py	Python script — open/edit with Notepad; RUN with `python`
agent-swarm/swarm/login_bootstrap.py	Python script — open/edit with Notepad; RUN with `python`
agent-swarm/swarm/merge_results.py	Python script — open/edit with Notepad; RUN with `python`
agent-swarm/swarm/preflight.py	Python script — open/edit with Notepad; RUN with `python`
agent-swarm/swarm/run_swarm.ps1	PowerShell script — RUN in PowerShell; view with Notepad
agent-swarm/swarm/run_swarm.sh	Shell script (Mac/Linux) — ignore on Windows
agent-swarm/swarm/run_worker.py	Python script — open/edit with Notepad; RUN with `python`
agent-swarm/work_queue.csv	Data — open with Excel (or Notepad)
agent-swarm/work_queue.json	Data — open with Notepad
citation-onboarding/COVER_NOTE.md	Text/Markdown — open with Notepad, or read it in this PDF
citation-onboarding/Citation_Bulk_Load.csv	Data — open with Excel (or Notepad)
citation-onboarding/Citation_Bulk_Load.xlsx	Spreadsheet — open with Excel
source- docs/Atlas_Migration_Ready_For_Approval_SUPERSEDED.txt	Text — open with Notepad
source- docs/Atlas_Training_Master_CQC_Compliant_SUPERSEDED.xlsx	Spreadsheet — open with Excel
source- docs/Course_Interval_Mapping_Reference_SUPERSEDED.xlsx	Spreadsheet — open with Excel

Project Index

_index.md

name: Atlas Training Migration

description: Migrating Select Lifestyles historical staff training records into Citation Atlas with CQC-compliant expiry intervals

type: project

status: active

last_updated: 2026-06-10

Atlas Training Migration — Select Lifestyles Limited

Migrating historical staff training records into **Citation Atlas** (care compliance system), with CQC-aligned expiry intervals applied so the team can track currency and renewals.

- **Organisation:** Select Lifestyles Limited (Learning Disability provider)
- **Target system:** Citation Atlas
- **Authority for intervals:** Skills for Care Statutory & Mandatory Training

Guide (Dec 2025); NICE SC1 / NG67; CQC Regulation 18 (Staffing)

CURRENT STAGE (as at 2026-06-10)

Verified master compiled — awaiting sign-off on open items, then Atlas entry.

We are **past a major correction**. An earlier "Ready for Approval" version (1,062 records, built from the Excel export alone) was **superseded** after a full audit of 102 paper attendance sheets revealed the Excel transcription had **stopped at 06/11/2025** — ~6 months of sessions (Dec 2025–May 2026) existed only on paper.

Authoritative file

➡ **Atlas_Training_Master_VERIFIED.xlsx** — this is the source of truth.

	Old (CQC_Compliant) ❌	VERIFIED ✅ (current)
Records	1,062	1,358
Unique staff	295	408 (+113 recovered)
Built from	Excel export only	Excel + 102 attendance sheets
Overdue	333	296

The old `_SUPERSEDED.` files in `source-docs/` are kept for history only —

do not sign those off or enter them into Atlas.

VERIFIED status breakdown (as at 04/06/2026)

- OVERDUE: **296**
- Expiring 0–30 days: 16
- Expiring 31–60 days: 7
- Current: 931
- No expiry: 108

What the audit recovered/fixed

- +417 person-records added from attendance sheets (missing from Excel)
- +113 staff who appeared only on 2026 sheets (e.g. May induction cohort)
- De-duplicated 7 duplicate scans
- Kept latest completion date per person+course (124 repeat/refresh pairs)
- Reconciled informal name variants vs HR spellings

OPEN ITEMS — need Elliot / HR decision before final Atlas entry

- [] **Unknown course** — PDF1 p25 (session 17/10/25) has no course name. Confirm what it was.
- [] **Uncertain handwritten surnames** — flagged for HR validation: Rimbould, Kaanom, Millican, Guash, Bastonoli, Salimatu Gesay, Oghenekharo (best-reading used; see Source column in master).
- [] **Interval-policy confirmation** — Makaton, Record Keeping, Risk Assessment, Driver Safety were **defaulted to 3-year** (not in core CQC list). Confirm against Select Lifestyles policy. (*Note: the superseded Mapping Reference treated some of these as "no expiry" — another reason it's stale.*)
- [] **Entry method decision** — Citation Atlas has **no self-serve bulk import** for historical records with expiry dates. Choose:
 - Manual "Add result" per record (slow for 1,358), or
 - Citation onboarding team bulk-load (need format from them).
- [] **Priority on entry** — address the **296 OVERDUE** records first.

CQC interval rules applied (in VERIFIED master)

- **1 year:** Medication, Buccal Midazolam, PEG, Insulin/Glucose, Diabetes,

Dysphagia, Epilepsy, MCA/DoLS, PRICE/PBS, BLS/CPR, Nurse Lead

- **3 years:** First Aid/EFAW, Moving & Handling, Safeguarding, Fire Safety,

Oliver McGowan/Autism, Infection Control, Food Hygiene, EDI

- **No expiry (one-time):** Induction, Care Certificate, Intensive Support
- **Assumption flag:** Makaton, Record Keeping, Risk Assessment, Driver Safety

→ defaulted to 3yr (see open item 3)

Two entry paths prepared

- **Citation onboarding bulk-load** (sanctioned, preferred) → `citation-onboarding/`

— clean import-ready `Citation_Bulk_Load.xlsx/ .csv` (1,351 records; 7 held)

plus `COVER_NOTE.md` with the questions Citation must answer (can they set arbitrary expiry dates, format, staff/course matching). Send this first.

- **Agent swarm** (DIY fallback) → `agent-swarm/` (see below). Only if Citation can't bulk-load in time, and subject to the ToS/authorisation check.

Runnable harness: `agent-swarm/swarm/` — parallel Playwright workers (`login_bootstrap.py` → `run_worker.py` → `merge_results.py`). Compiles and runs in `--dry-run`; needs the form selectors from one authenticated walkthrough + a login session before a live run. Start at `agent-swarm/swarm/README.md`.

Agent-swarm entry package

→ `agent-swarm/` — everything needed to drive a swarm of browser agents to enter the 1,358 records into the Atlas portal. Start at `agent-swarm/ORCHESTRATION.md`. Contains: claimable `work_queue.csv` (1,358 rows, priority-sorted, 7 HOLD + 168 interval-caution pre-flagged), per-agent runbook, field mapping, exceptions, QA/reconciliation, and a portal-walkthrough template to capture the real selectors before launch.

Files in this folder

- `Atlas_Training_Master_VERIFIED.xlsx` —  authoritative master (Summary, Discrepancy Report, Atlas Training Master sheets)

- `agent-swarm/` — swarm entry package (see above)
- `source-docs/` — superseded history:
- `Atlas_Training_Master_CQC_Compliant_SUPERSEDED.xlsx`
- `Course_Interval_Mapping_Reference_SUPERSEDED.xlsx`
- `Atlas_Migration_Ready_For_Approval_SUPERSEDED.txt`

Next session pickup

Start here. The two highest-leverage moves are confirming the ****interval policy** (open item 3) **and the** unknown course (open item 1)**, since both alter the master before it goes into Atlas. Then decide entry method and work the 296 OVERDUE records first.

Safety Protocol

SAFETY_PROTOCOL . md

SAFETY PROTOCOL — operating the live Atlas portal

This is a **live production care-compliance system**. The goal: every record entered **correctly**, nothing **mass-applied**, nothing ****edited** or deleted by accident, **and a way to prove and reverse**** anything that goes wrong. Read this before any automated or assisted entry. The rules here override speed.

0. The 6 golden rules (non-negotiable)

- **ONLY ever add training.** Use the **Add Training** panel and the **Add** button. Never Edit, Delete, Remove, Archive, Import, "Update Employees", or any bulk action.
 - **"Add training to" = Employee. ALWAYS.** Never select **All employees in the company, Site, Employee group, or Department or team**. Those apply one course to **many people at once** — the single most dangerous action on this screen. If "Employee" isn't selected, **do not click Add**.
 - **One named person per record.** Confirm the specific person is selected before saving. No person selected, or scope not "Employee" → **stop**.
 - **Never invent or guess.** Wrong/unknown person, course, or date → log ERROR and move on. Accuracy beats completeness.
 - **Verify every save.** A record isn't "done" until you've seen it saved against the **right person**. Don't assume.
 - **When unsure, stop and ask a human.** Any unexpected dialog, confirmation prompt, or destructive-looking button → pause, don't click through.
-

1. Before you start (do these once)

- **[] EXPORT a backup first.** On Employees → Manage, the  menu has **EXPORT** — export the current employee/training data and save it with today's date. This is your "before" snapshot and your rollback reference. ****Do not skip this.****

- [] **Confirm the right organisation.** Top-right should read ****Elliot Horton**

/ Select Lifestyles Limited**, URL contains the org id

9892b03b-...576d96. If it shows a different company, stop.

- [] **Close other editors.** No one else (and no other tab/agent) should be

editing Atlas at the same time — concurrent edits cause collisions and duplicates.

- [] **Single operator.** One agent / one session only. No parallel swarm

against this portal (it collides on one login and trips bot-detection).

2. The catastrophic risks, and how each is blocked

Risk	Consequence	Control
Pick a bulk scope ("All employees" / Department)	One course applied to 408 staff; reversal = deleting hundreds	Rule 2: Employee scope only, verified before every Add
Click Delete / Remove on an existing record	Real training history lost	Rule 1: never touch delete controls; add-only
Edit/overwrite an existing entry	Existing record corrupted	Rule 1: only the Add Training panel; never open/modify existing records
Use IMPORT / Update Employees	Mass overwrite of employee data	Rule 1: those : items are off-limits
Wrong person via autocomplete	Training on the wrong file	Rule 5: confirm the exact name before save
Date flips (mm/dd)	Wrong expiry / false compliance	§4 date check; the 13/11/2024 test
Duplicate entries	Inflated/again records	Dedupe check before each save
Silent save failure (network)	Thinks done, isn't	Rule 5: confirm the record appears
Session expiry mid-run	Partial/failed saves unnoticed	§5 batch checkpoints; re-login and re-verify

Key reassurance: because we are **add-only** and never use delete, the

worst realistic accident is *extra/wrong records* — which are identifiable in

the log and removable deliberately — not silent loss of existing history. The

one exception is bulk-scope (Rule 2), which is why that rule is absolute.

3. Per-record verification (every single record)

- Scope shows **Employee**, and the **correct named person** is selected.
- Course is the intended course (closest exact match; never a guess).
- Start = Completed = the record's completed date; Expiry = the record's expiry

(blank for no-expiry courses).

- Click **Add**; **see the confirmation** / panel close.
- Mark the outcome in `entry_log.csv` (DONE / SKIPPED / HOLD / ERROR + note).
- Periodically (and always for the first few), open the person's **Training**

list and confirm the new record is there with the **right dates**.

4. Dates

UK **DD/MM/YYYY**. First proof is **ATL-0006 = 13/11/2024 → must read as 13

November 2024. **If Atlas rejects it, the field wants another format** — stop and

flag it **before continuing**. **Also confirm the** expiry you typed is what's

stored** (if Atlas recalculates expiry from the course, your value may be

overridden — check on the pilot and report).

5. Work in batches with checkpoints

- **Batch 1: the 296 OVERDUE only** (rows 1–296). **Stop and report** the tally

before going further. Spot-check ~5 of them in Atlas.

- Then proceed in **blocks of ~100–200**, pausing to report after each.
- This keeps sessions fresh, catches any systematic error early (before it's

repeated 1,000 times), and is easy to resume across sittings.

- **Don't run unattended** for the first batch — watch the first ~10 live.

6. Stop immediately and tell a human if...

- Scope won't stay on "Employee", or a person can't be uniquely identified.
- Any **Delete/confirm/"are you sure"** dialog appears.
- A **CAPTCHA** or **"unusual activity"** warning.
- You're **logged out** or asked to re-authenticate.
- The **13/11/2024** date won't save.
- A course is missing for **many** records (needs an admin to add it).
- Anything behaves differently from this protocol.

7. After each batch — recording & reconciliation

- Keep `entry_log.csv` current (every `record_id` has an outcome).
- After a batch, **EXPORT from Atlas again** and compare the new training count

to your log — the increase should equal your DONE count for the batch.

- **Final reconciliation:** `DONE + SKIPPED + HOLD(7) + ERROR = 1,358`, and an

Atlas export of training records matches the master for the people entered.

8. If something goes wrong (recovery)

- **Stop**. Don't try to "fix it fast" with more clicks.
- Use `entry_log.csv` to see exactly what was entered and when.
- Compare against the **pre-start export** (§1) to identify any unintended

change.

- If bulk-scope was triggered: list the affected staff from the export diff and remove those specific erroneous records deliberately (or ask Citation support) — don't bulk-delete blindly.
 - Document what happened so the master/log stays the source of truth.
-

One-line summary for the operator: *Add-only, Employee-scope-only, verify every save, export a backup first, work the 296 OVERDUE as batch 1 and stop to report — and when in doubt, don't click, ask.*

Claude-in-Chrome Handover

CLAUDE_CHROME_HANDOVER.md

HANDOVER — Atlas Training Migration (Claude in Chrome)

Full operating brief. Paste this whole file into the Claude panel in the

Atlas browser tab, attach `work_queue.csv`, and tell it to begin.

0. SAFETY — READ FIRST (non-negotiable; see SAFETY_PROTOCOL.md)

This is a **live production** care-compliance system.

- **Add-only.** Use the Add Training panel + **Add** button. **Never** Edit, Delete, Remove, Import, or "Update Employees" — and never click any delete/remove icon on an existing record.
- **"Add training to" = Employee, ALWAYS.** Never pick *All employees*, *Site*, *Employee group*, or *Department or team* — those apply a course to ****many** people at once** and are extremely hard to undo. If "Employee" is not selected, **do not click Add**.
- **One named, correct person per record.** Verify the person before saving.
- **Verify every save** (right person, right dates). Don't assume.
- **Export a backup first** (: → EXPORT) before entering anything.
- **When unsure, STOP and ask Elliot.** Any "are you sure" / delete /

confirmation dialog → do not click through.

1. Who you are / what you're doing

You are operating **inside Elliot's logged-in Citation Atlas tab**

(`hrhs.atlas-hub.co.uk`, org: **Select Lifestyles Limited**, a Learning Disability provider). You can see the page — work **visually**, you don't need CSS selectors. Your job: enter **1,358 historical staff training records** into Atlas through the **"Add Training"** form, one record at a time, accurately, and without creating duplicates or wrong people.

This is a CQC-compliance migration: the records are real training that staff have done; Atlas needs them so currency/expiry tracking is correct.

2. The data

- Source: `work_queue.csv` (attached). 1,358 rows, already sorted with the

296 OVERDUE records first, then expiring, then current, then no-expiry.

Columns: ``record_id, status, training_course, full_name, completed_date, expiry_date, has_expiry, flag`` (plus a few you can ignore).

- **Dates are DD/MM/YYYY (UK).** 09/09/2024 = 9 September 2024.
- A second file, `entry_log.csv`, lists every `record_id` with blank

`outcome / atlas_note` columns — fill it in as you go (this is the audit trail and the reconciliation at the end).

3. Where the form is

Left sidebar **Employees / Workers** → **Manage** → top-right ⋮ (the three-dots button next to the pink "Add Employee / Worker") → **Add Training**.

A side panel titled "Add training history details" opens.

4. Per-record procedure (repeat for each row, in file order)

- Open the **Add Training** panel (⋮ → Add Training).
- **"Add training to"** → select **Employee**.
- **Employee** box → type `full_name`, wait for suggestions, click the matching

person.

- If it shows **"No Result Found"** → outcome = **ERROR**, note "staff not found", close the panel, move on. **Never create a new employee.**

- If several people share the name and you can't tell which → **ERROR**, "ambiguous staff", move on.

- **Course** → type `training_course`, pick the matching course.
- No match in Atlas's list → **ERROR**, "course missing", move on. Don't

guess or pick a near-name. (See §7 on course-name differences.)

- **Start date** → `completed_date`. (We only hold a completion date, and Start

is required, so use the completion date here too.)

- **Completed date** → `completed_date`.
- **Expiry date** → `expiry_date`.
- If `has_expiry` = **NO** (a one-time course), **leave Expiry blank**.
- Leave Course code / CPD / grade / provider / description **blank**.
- Click **Add**. Confirm the panel closes / a success message shows.
- Write the outcome in `entry_log.csv` against this `record_id`:

DONE / SKIPPED / HOLD / ERROR + a short note.

5. Hard rules

- **HOLD — skip these 7 records** (uncertain handwritten surnames; entering them

risks creating a wrong/duplicate staff member). Mark each **HOLD**, don't

enter:

record_id	name	course
ATL-0106	Amy Bastonoli	PRICE (PBS)
ATL-0130	Ann Guash	PRICE (PBS)
ATL-0340	David Rimbould	Moving & Handling
ATL-0432	Fatima Kaanom	Moving & Handling
ATL-0561	Jade Millican	Moving & Handling
ATL-1124	Salimatu Gesay	Record Keeping
ATL-0930	Oghenekharo	Induction

(They're also tagged `flag = HOLD-HR-NAME` in the data.)

- **No duplicates.** Before saving, if Atlas already shows that ****person + course with the same completed date, don't add it again → SKIPPED****, note "already in Atlas".

- **Never invent** a name, course, or date. Unsure about anything → **ERROR**, note why, move on. Accuracy beats completeness.

- **Order:** go top-to-bottom in the file (OVERDUE first). Don't skip ahead.
- **Pace:** a few seconds per record; don't machine-gun it.

6. Dates — important check

The format is **DD/MM/YYYY**. The first unambiguous test is **ATL-0006**:

`13/11/2024` must save as **13 November 2024**. If Atlas rejects it (treats 13 as a month), the date field wants a different format — **stop and tell Elliot** before doing more.

7. Course-name differences

Our course labels may not match Atlas's library exactly (e.g. "PRICE (PBS)", "Moving & Handling", "MCA / DoLS", "First Aid"). Pick the course in Atlas that clearly corresponds. If there's genuinely no matching course in Atlas, that course needs creating by an admin — log those records **ERROR "course missing"** and list the course names for Elliot rather than forcing a wrong match.

8. Pause-and-tell-Elliot conditions

- A **CAPTCHA** or "**unusual activity**" warning.
- You get **logged out** / asked to re-authenticate.
- The **13/11/2024** date won't save (format issue).
- A course is missing for many records (needs admin to add it).
- Anything that would require guessing a person's identity.

9. Work in batches (don't do all 1,358 in one go)

- **Batch 1 = the 296 OVERDUE only (rows 1–296). Then STOP and report**, and

wait for Elliot to spot-check ~5 in Atlas before continuing.

- After that, proceed in **blocks of ~100–200**, pausing to report after each.
- This catches any systematic mistake early (before it repeats 1,000 times) and

lets the work resume across sittings. Don't run unattended on Batch 1 — the

first ~10 should be watched live.

10. Reporting & reconciliation

- After **every 25 records**, post a running tally:

DONE / SKIPPED / HOLD / ERROR, plus the `record_ids` of any ERRORS.

- Keep `entry_log.csv` updated so the final state is auditable.
- After each batch, **EXPORT from Atlas** and confirm the training count rose by

your DONE count (catches silent failures).

- **Target reconciliation:** `DONE + SKIPPED + HOLD(7) + ERROR = 1,358`.

10. Authority for the data (already agreed — no need to re-decide)

Intervals from Skills for Care Statutory & Mandatory Training Guide (2025) +

NICE SC1/NG67 + CQC Regulation 18. Makaton, Record Keeping, Risk Assessment,

Driver Safety = **3-year** (signed off by Elliot). 107 one-time courses

(induction, Care Certificate, etc.) have **no expiry**. Expiry dates are already

calculated in the data — just enter them.

11. Start here (first 3, report each before continuing)

record_id	full_name	course	completed	expiry
ATL-0001	Abduli Msongo	Epilepsy	09/09/2024	09/09/2025
ATL-0003	Abduli Msongo	Medication	09/09/2024	09/09/2025
ATL-0006	Abdulrahmon Ayo Hussein	PRICE (PBS)	13/11/2024	13/11/2025

Do these three, confirm they saved correctly in Atlas (open the person's

Training to verify the dates), report the results, then continue through the file.

Approval & Go-Live

APPROVAL .md

name: Atlas Migration — Approval & Go-Live

type: project

status: awaiting-sign-off

date: 2026-06-10

Atlas Training Migration — Approval & Go-Live Sheet

Select Lifestyles Limited (Learning Disability provider) → Citation Atlas

Sign this off to authorise entry of the verified training data into Atlas via the swarm. Everything referenced is in this folder.

1. What will be entered

- **1,358 verified training records**, 408 staff, ~34 courses.
- Source of truth: `Atlas_Training_Master_VERIFIED.xlsx` (Excel export + a

102-page paper attendance-sheet audit that recovered 417 records / 113 staff

the spreadsheet had dropped after 06/11/2025).

- Status mix: **296 OVERDUE** (entered first), 23 expiring, 931 current,

108 no-expiry.

- Of these, **7 are HELD** (uncertain handwritten surnames) and entered later —

so the swarm enters **1,351** now.

2. How it will be entered

- **Method:** a swarm of parallel scripted Playwright workers (`agent-swarm/swarm/`)

reusing ONE human login session. Deterministic, throttled (8–15s/record),

dedupe-guarded, screenshot-audited.

- **Priority:** OVERDUE block (rows 1–296) first, then the rest.
- **You watch it live:** each record prints `record_id · name · course → result`

to the console as it goes in; a screenshot is saved per record.

3. Decisions needed BEFORE go-live (tick to approve)

- ☐ **Data is correct** — the 296 OVERDUE and overall figures match expectation.
- ☒ **Interval policy** — Makaton, Record Keeping, Risk Assessment, Driver

Safety (168 records) default to **3-year** expiry. **CONFIRMED 3yr (Elliot, 2026-06-10).**

- ☐ **Unknown course** — PDF1 p25 (17/10/2025): identify or accept it's excluded.
- ☐ **7 HELD names** — accept they're entered later once HR confirms spelling.
- ☐ **Authorisation / ToS** — automated owner-driven entry into Atlas is

acceptable (or it will be run as *assisted* entry with a human present).

- ☐ **Method** — proceed with swarm (vs. Citation onboarding bulk-load, which

remains the lower-risk option — see `citation-onboarding/`).

Approver: _____ Date: _____ Signature: _____

4. Two human-gated unlocks (cannot be automated — login/MFA)

- **Selector walkthrough** — do one real "Add result" in Atlas and fill the

TODO selectors in `agent-swarm/swarm/config.py`

(guide: `agent-swarm/PORTAL_WALKTHROUGH_TEMPLATE.md`). ~10 min, once.

- **Login session** — `python agent-swarm/swarm/login_bootstrap.py`, log in

with MFA, session saved. Re-run when it expires.

5. Go-live (after sign-off + the two unlocks)

```
cd "agent-swarm/swarm"
python preflight.py                # must read READY ☒
python run_worker.py --worker pilot --start 1 --end 10 # 10-record pilot
# -> verify those 10 in Atlas (QA_VERIFICATION.md), then:
./run_swarm.sh 1 296 5             # the 296 OVERDUE across 5 workers (watch live)
./run_swarm.sh 297 1358 5          # the remainder
python merge_results.py            # reconcile -> must total 1,358
```

6. Done = signed off when

- `merge_results.py` reconciles: `DONE + SKIPPED + HELD(7) + ERROR = 1,358`.
- All 296 OVERDUE accounted for; ERRORS itemised with an owner.
- Screenshots archived. `_index.md` status updated to "Atlas entry complete".

Current readiness (auto-checked 2026-06-10)

- ☒ Data verified & queued (1,358) · ☒ Swarm harness built & dry-run passing ·

☒ Citation bulk-load package prepared as fallback

- ⌚ Awaiting: this sign-off · selector walkthrough · login session

Form Fields / Expiry Finding

agent-swarm/EXPIRY_FINDING.md

Add Training form — confirmed fields (2026-06-10)

Corrects an earlier note: the form **does** have an expiry field (it sits below the fold; the first selector capture stopped short of it). Good news — we enter our own expiry per record and don't depend on course config.

Full field list (the "Add training history details" panel)

- **Add training to** * (#ddlDistributeTo) — dropdown:

*Please select / All employees in the company / Site / Employee group /

Department or team / **Employee***. → choose **Employee**, then a second

control appears to pick the specific person.

- **Course** * (#SelectedCourse) — autocomplete: type then select.
- Course code (#CourseCode) — optional, skip.
- CPD Minutes (#CPDMinutes) — optional, skip.
- **Start date** * (#add-training-bulk-mode_AeDatetimePicker_1_ae-input_3)
- **Completed date** (#add-training-bulk-mode_AeDatetimePicker_2_ae-input_3)
- **Expiry date** (datepicker — selector to confirm, likely

#add-training-bulk-mode_AeDatetimePicker_3_ae-input_3)

- Course grade / Provider / Description / Certificates — optional, skip.
- **Add** (submit) #add-training-bulk-mode_AeButton_1_aeButton_1 · Close.

Per-record mapping (migration)

Form field	Our data
Add training to	"Employee" → then pick full_name
Course	training_course (autocomplete)
Start date *	completed_date (we only hold completion; use it for both)
Completed date	completed_date
Expiry date	expiry_date (blank for the 107 no-expiry records)

Start date is required and we have no separate "started" date for historical training, so Start = Completed = our completion date. Expiry comes straight from our master, so course-interval config in Atlas is **not** a dependency.

Still to confirm on the first pilot record (3 selectors)

- The **employee picker** that appears after choosing "Employee" (and how to select by name).

- The **course autocomplete option** element.
- The **expiry date** input id (confirm the `_3_` guess) and the **success** confirmation after Add.

Exceptions

agent-swarm/EXCEPTIONS.md

Exceptions & Unresolved Items

These are pre-flagged in `work_queue.csv` (the `flag` column). Resolve or explicitly accept before/while the swarm runs.

1. HOLD — uncertain surnames (7 records, `flag=HOLD-HR-NAME`)

Handwritten 2026 surnames read with low confidence. Entering an unverified spelling risks creating a **duplicate / wrong staff record** in Atlas.

Affected name fragments: ****Rimbould, Kaanom, Millican, Guash, Bastonoli, Salimatu Gesay, Oghenekharo****.

→ **Action:** agents set `entry_status=HOLD` and skip. HR confirms the correct HR-system spelling, then these are entered manually (or un-flagged in the queue and re-run). Until then they stay out of Atlas.

2. CAUTION — unconfirmed interval (168 records, `flag=CAUTION-INTERVAL`)

Courses **Makaton, Record Keeping, Risk Assessment, Driver Safety** are not in the core CQC list and were **defaulted to 3-year** expiry. The superseded mapping doc treated some as "no expiry" — so the expiry dates on these rows are an assumption.

→ **Action:** confirm the intended interval against Select Lifestyles policy.

- If 3-year stands → enter as-is (agents already do; expiry is in the row).
- If they should be "no expiry" or 1-year → fix `interval_yrs/expiry_date`

in the queue first, *then* enter. Don't let agents enter the wrong expiry and have to redo 168 records.

3. Unknown course — PDF1 p25 (17/10/2025)

One attendance session had no course name. It is **not** represented as an enterable row (course unknown). → Confirm what the session was; add to the queue only once identified.

4. Course-name → Atlas mapping (see `FIELD_MAPPING.md`)

Any master course label that doesn't exactly match an Atlas course must be mapped

(or the course created in Atlas by Citation) before those rows can be entered.

Agents ERROR rather than guess.

Reconciliation math

1,358 total = entered (DONE) + dedupe-skips (SKIPPED) + 7 HOLD + any ERROR.

At sign-off, ERROR and HOLD counts should be explained, not silently dropped.

Swarm Orchestration

agent-swarm/ORCHESTRATION.md

Atlas Swarm — START HERE (Orchestration)

Goal: enter 1,358 verified training records into the Citation Atlas web portal using a swarm of browser-driving agents, in parallel, without collisions or duplicates.

> Authoritative data: `../Atlas_Training_Master_VERIFIED.xlsx`

> Work queue: `work_queue.csv` (and `.json`) — the single source agents read/write.

⚠ Read before you launch (4 blockers)

- **Atlas has no self-serve bulk import** for historical records with expiry dates (per Citation). The *sanctioned* fast path is Citation's onboarding team bulk-loading a spreadsheet. A browser swarm is the DIY alternative — use it only if Citation can't/won't bulk-load in your timeframe.
- **Authorisation / ToS.** Automating a third-party SaaS with a swarm may breach Atlas terms and can trip bot-detection (CAPTCHA, rate-limits, account lock). Confirm with Citation that automated entry by the account owner is acceptable, or treat the agents as "assisted manual entry" (human logged in, agent fills forms) rather than headless bots.
- **Credentials never go in these files or agent prompts.** A human logs into Atlas and hands each agent an *already-authenticated browser session*. Agents must never see, type, or store the Atlas password/MFA.
- **Open data items still unresolved** (see `EXCEPTIONS.md`). 7 records are on HOLD (uncertain surnames — entering them risks duplicate staff in Atlas) and 168 carry an unconfirmed interval. These are pre-flagged in `work_queue.csv`.
If any of 1–4 is a no, stop and resolve it before spinning up the swarm.

The documents in this folder

File	Purpose
<code>ORCHESTRATION.md</code> (this file)	How the swarm is partitioned, claims work, avoids dupes, and reports progress

File	Purpose
AGENT_RUNBOOK.md	The per-record SOP — give this to each agent verbatim as its instructions
FIELD_MAPPING.md	Source column → Atlas form field, with exact value transforms
EXCEPTIONS.md	Records to HOLD/skip and the unresolved policy questions
QA_VERIFICATION.md	Post-entry verification + reconciliation checklist
work_queue.csv / .json	The 1,358-record claimable task list (source of truth for progress)
PORTAL_WALKTHROUGH_TEMPLATE.md	Fill this in from ONE manual entry to capture the real selectors before launch
LOGIN_FLOW.md	Azure B2C login flow — confirmed step-1 selectors, expected password/MFA, session-reuse strategy
portal-captures/	Live screenshots/DOM captures of the portal (login page captured 2026-06-10)

Step 0 — Capture the real portal flow (do this ONCE, manually)

The runbook below is correct on *logic* but cannot contain real CSS selectors /

field labels until someone records one real entry. A human (or one agent with a

human watching) does a single "Add result" in Atlas and fills in

PORTAL_WALKTHROUGH_TEMPLATE.md: the URL, the exact field labels, dropdown

behaviour (does it autocomplete staff names? create-new vs select-existing?),

the date format the field accepts, and the save/confirm step. **Do not launch

the swarm until that template is filled** — otherwise every agent fails

identically.

Step 1 — Partition the work (sharding)

Agents must never work the same record. Shard by **record_id ranges** taken

in **priority order** (the CSV is already sorted: OVERDUE → EXPIRING → CURRENT →

NO EXPIRY). Recommended: start with a **small pilot** then scale.

- **Pilot:** 1 agent, first 10 OVERDUE rows. Verify in Atlas. Fix the runbook.
- **Scale:** N agents (start N=3–5, not 50). Lower N = lower lock/ban risk.

Two collision-safe options:

- **Static shards (simplest):** split the queue into N contiguous blocks and

hand each agent its block (e.g. Agent A = rows 1–60, B = 61–120 ...). No shared

state needed; just don't overlap ranges. Process OVERDUE block first.

- **Dynamic claim (resilient):** agents atomically claim the next PENDING row

by writing their id into `claimed_by` + setting `entry_status=IN_PROGRESS`

before acting. Requires a lock so two agents don't grab the same row — see

"Claiming protocol" below.

For ≤ 5 agents, **static shards are simpler and safe**. Use dynamic claim only if you want fault-tolerance (an agent dying mid-block).

Step 2 — Claiming protocol (only if using dynamic claim)

`work_queue.csv` is not concurrency-safe if many writers hit it at once. Use ONE coordinator:

- A single **coordinator agent/process** owns the CSV. Workers ask it "give me the next record"; it marks `IN_PROGRESS` + `claimed_by` and hands it over.
- On success worker reports back → coordinator sets `DONE` + writes the Atlas confirmation id into `atlas_confirmation`.
- On failure → `ERROR` with reason in `agent_notes`; coordinator re-queues or parks it.
- Never let workers write the shared CSV directly in parallel.

Step 3 — The dedupe rule (critical)

One record = unique (`full_name` + `training_course` + `completed_date`).

Before saving, the agent searches Atlas for that staff+course; if a result with the same completed date already exists, it **SKIPS** (`entry_status=SKIPPED`, note "already in Atlas") rather than creating a duplicate. The master already de-duplicated and kept the latest completion per person+course, so within the queue each row is unique — this guard catches re-runs and prior manual entries.

Step 4 — Rate & safety limits

- Throttle: ~1 record / 8–15s per agent (human-like). Don't hammer.
- Hard stop on: repeated CAPTCHA, "unusual activity" warning, session logout, or 3 consecutive save failures → pause that agent, alert the human.
- Each agent keeps an append-only log (`logs/agent-<id>.log`): `record_id`, timestamp, action, result, Atlas confirmation/screenshot path.
- Screenshot the Atlas confirmation after each save (audit trail).

Step 5 — Reconcile (see `QA_VERIFICATION.md`)

When `entry_status` is all `DONE`/`SKIPPED`/`HOLD`: pull an Atlas training export and diff against `work_queue.csv` — counts must reconcile (entered + skipped-dupes +

held = 1,358). Spot-check a sample of OVERDUE entries for correct dates/expiry.

Suggested launch order

- Fill `PORTAL_WALKTHROUGH_TEMPLATE.md` from one manual entry.
- Resolve / accept the `EXCEPTIONS.md` items (or accept HOLD on the 7).
- Pilot: 1 agent × 10 OVERDUE rows → QA → tune runbook.
- Scale to 3–5 agents, OVERDUE block first (296 rows), then the rest.
- Reconcile and sign off.

Agent Runbook

agent-swarm/AGENT_RUNBOOK.md

Atlas Agent Runbook (per-record SOP)

- > Give this to each swarm agent as its operating instructions. It describes how
- > to enter **one** record into the Citation Atlas portal correctly and safely.
- > UI specifics marked `<CONFIRM>` come from `PORTAL_WALKTHROUGH_TEMPLATE.md` —
- > fill those in before launch.

Your role

You enter training records into the Atlas web portal, one at a time, from the work queue. You are precise, you never guess, and you never create duplicates. You operate inside a browser session a human has **already logged in** — you do not handle credentials or MFA.

Inputs per record (from `work_queue.csv`)

`record\_id, status, training\_course, full\_name, completed\_date, expiry\_date, interval\_yrs, has\_expiry, flag, ...`

Hard rules

- **Never enter a row whose `flag = HOLD-HR-NAME`.** Set `entry_status=HOLD`, move on. (Uncertain surname — entering it risks a duplicate staff record.)
- **`flag = CAUTION-INTERVAL`** → enter normally, but the expiry derives from an *unconfirmed* interval. Enter it; note `interval-unconfirmed` in `agent_notes`.
- **Dedupe before save** (Step in the procedure). If it already exists → SKIP.
- **Never invent data.** If a staff name isn't found in Atlas, or a course isn't in the Atlas course list, or a date won't accept → STOP that record, set `entry_status=ERROR` with the reason; do not improvise.
- **No credentials, ever.** If logged out or MFA prompts → pause, alert human.
- Respect the throttle (~1 record / 8–15s). Stop on CAPTCHA / "unusual activity" / 3 consecutive failures.

Procedure (one record)

- **Claim** the record (mark `IN_PROGRESS`, write your agent id). Skip if

using static shards.

- **Check flag** → HOLD? set HOLD and stop. CAUTION? continue.
- **Navigate** to <CONFIRM: Atlas → Training Reports → "Add result"> .
- **Select staff member:** type `full_name` into <CONFIRM: staff field>.
- Must **match an existing** staff record. If multiple/none match → ERROR

"staff not found / ambiguous", stop. Do NOT create a new staff member.

- **Select training course:** choose `training_course` from <CONFIRM: course

dropdown>. Use the exact Atlas course name per `FIELD_MAPPING.md` (some

master names differ from Atlas labels). Not in list → ERROR "course missing".

- **Dedupe check:** look at existing results for this staff+course. If one

exists with the same `completed_date` → set SKIPPED ("already in Atlas"),

stop.

- **Completed/Date taken:** enter `completed_date` in <CONFIRM: date format,

e.g. DD/MM/YYYY>. Source dates are DD/MM/YYYY — convert if the field

differs.

- **Expiry date:**

- `has_expiry = YES` → enter `expiry_date`. If Atlas auto-calculates expiry

from the course interval, verify it matches `expiry_date`; if it differs,

prefer the master `expiry_date` (it's the agreed value) and note the delta.

- `has_expiry = NO` → leave expiry blank (one-time course). <CONFIRM how Atlas

represents no-expiry>.

- **Status/result:** <CONFIRM whether Atlas derives status from dates or needs

it set>. If it derives automatically, do nothing; if manual, map per

`FIELD_MAPPING.md`.

- **Save** <CONFIRM save button>. Wait for the success confirmation.
- **Capture proof:** screenshot the confirmation → `logs/<record_id>.png`.

Record the Atlas confirmation/reference into `atlas_confirmation`.

- **Report:** set `entry_status=DONE`; append to your agent log

(`record_id`, timestamp, result, screenshot path).

- Throttle, then next record.

Outcome states

- `DONE` — saved + confirmed in Atlas.

- SKIPPED — already present (dedupe).
- HOLD — flagged HOLD-HR-NAME, not entered.
- ERROR — could not complete; reason in agent_notes (staff not found,

course missing, date rejected, save failed, session lost).

Stop conditions (pause + alert human)

CAPTCHA · "unusual activity"/rate warning · logged out / MFA · 3 consecutive

ERRORS · any prompt to change account settings or anything outside "add a training result".

Field Mapping

agent-swarm/FIELD_MAPPING.md

Field Mapping — Master → Atlas form

Source: `work_queue.csv` (derived from `Atlas_Training_Master_VERIFIED.xlsx`).

Atlas field names marked `<CONFIRM>` until `PORTAL_WALKTHROUGH_TEMPLATE.md` is done.

Master / queue column	Atlas form field <code><CONFIRM></code>	Transform / rule
<code>full_name</code>	Staff member (select existing)	Match existing staff. Never create new. See name notes below.
<code>training_course</code>	Training course (dropdown)	Use Atlas label per course table below.
<code>completed_date</code>	Completed / Date taken	Source is <code>DD/MM/YYYY</code> . Reformat to the field's format.
<code>expiry_date</code>	Expiry / Renewal date	Only if <code>has_expiry=YES</code> . Leave blank for one-time courses.
<code>interval_yrs</code>	(usually not entered)	Informational. If Atlas sets expiry from course interval, verify it equals <code>expiry_date</code> ; prefer master value on mismatch.
<code>status</code>	(usually derived)	If Atlas derives from dates, do nothing. Else: <code>OVERDUE/EXPIRING/CURRENT</code> map to Atlas's equivalent labels.
<code>source</code>	—	Internal provenance, not entered.

Value rules

- **Dates:** master uses `DD/MM/YYYY`. Confirm Atlas's accepted format and

convert consistently. Watch for US `MM/DD` traps on ambiguous dates.

- **No-expiry (108 records):** `has_expiry=NO`, `interval_yrs` blank,

`status NO EXPIRY` — one-time courses (Induction, Care Certificate, Intensive

Support, plus other induction-type subjects). Leave expiry empty.

- **Status values present:** `OVERDUE` (296), `EXPIRING` (0-30d) (16),

`EXPIRING` (31-60d) (7), `CURRENT` (931), `NO EXPIRY` (108).

Course-name reconciliation

The master uses these 34 course labels. Before launch, map any that don't match

the Atlas course list **exactly** (Atlas selection must hit an existing course;

a missing course → `ERROR`, not a guess). Known abbreviations to verify in Atlas:

- `PRICE` (PBS) → confirm Atlas label (PRICE / Positive Behaviour Support / PBS)
- `First Aid` → EFAW / First Aid at Work?

- Medication → Safe Handling & Administration of Medication / Management

Medication Level 2? (master may have collapsed variants — check)

- MCA/DoLS, Buccal Midazolam, PEG, Moving & Handling, `Oliver McGowan /

Autism, Infection Control, EDI, Food Hygiene` → confirm exact Atlas text.

> Action: open the Atlas course dropdown once, export/screenshot the full course

> list, and complete a master course → Atlas course lookup table here. Any

> master course with no Atlas equivalent must be raised with Citation (course

> created in Atlas) before those rows can be entered.

Name notes

- Master keeps HR/official spellings where known; informal variants were already

reconciled (Renae → Renee, Jo → Joanne, Viv → Vivienne, Elliott → Elliot,

Hazelhurst → Hazlehurst).

- **7 rows flagged HOLD-HR-NAME** (uncertain handwritten surnames) must NOT be

entered until HR confirms spelling — see EXCEPTIONS.md.

QA & Verification

agent-swarm/QA_VERIFICATION.md

QA & Verification

Run during the pilot and again at the end. The swarm isn't "done" until this reconciles.

Per-record (every agent, every save)

- [] Atlas confirmation/success message seen and screenshotted → `logs/<record_id>.png`
- [] Staff name matched an **existing** record (no new staff created)
- [] Course matched an existing Atlas course (exact)
- [] Completed date entered = `completed_date` (correct DD/MM, no US flip)
- [] Expiry = `expiry_date` (or blank for `has_expiry=NO`); if Atlas auto-set a

different expiry, the delta is noted

- [] `entry_status` + `atlas_confirmation` written back to the queue

Pilot gate (after first 10 OVERDUE)

- [] Open each of the 10 in Atlas UI and eyeball-confirm fields
- [] No duplicates created; no wrong-staff matches
- [] Runbook `<CONFIRM>` placeholders all replaced with real values
- [] Date format verified on a deliberately ambiguous date (e.g. 03/04/2025)
- → Only then scale up.

Final reconciliation

- Export the training results from Atlas (or pull a report).
- Diff against `work_queue.csv`:
- `DONE` rows present in Atlas with matching staff/course/completed/expiry
- `SKIPPED` rows correspond to a pre-existing Atlas entry (legit dupe)
- `HOLD` = 7 (the uncertain surnames), still out
- `ERROR` rows itemised with reason and a remediation owner
- **Count check:** `DONE + SKIPPED + HOLD + ERROR = 1,358`.
- Spot-check a random 30 (weighted to OVERDUE) for field accuracy.
- Confirm the **296 OVERDUE** are all entered/accounted for (priority block).

Sign-off

- [] Counts reconcile to 1,358
- [] OVERDUE block fully accounted for
- [] HOLD/ERROR lists have named follow-up owners
- [] Screenshots/logs archived in `logs/`
- [] Update `../_index.md` status → "Atlas entry complete (date), N entered,

M skipped-dupes, 7 held, K errors"

Login Flow

agent-swarm/LOGIN_FLOW.md

Atlas Login Flow (Azure AD B2C)

Captured live via Playwright on 2026-06-10. The portal redirects unauthenticated

users to a Microsoft **Azure AD B2C** sign-in. Screenshot:

`portal-captures/atlas-login.png`.

> Needed by the swarm to reach an authenticated state. **Credentials are never

> stored here or given to agents** — a human establishes the session; agents

> inherit it.

Entry point

- App URL (deep link): `https://hrhs.atlas-hub.co.uk/o/9892b03b-455e-4b80-8490-a76073576d96/employee/manage`
- Redirects to auth host: `https://auth.atlas-hub.co.uk/atlashub.onmicrosoft.com/b2c_1a_rest_signup_signin_auth/oauth2/v2.0/authorize?...`
- Auth type: Azure AD B2C (OAuth2 PKCE, `response_type=code`, MSAL.js 3.21.0)
- `client_id=86bf93bb-2d13-4286-a9a7-606c2a0e95b8`, `redirect_uri=https://hrhs.atlas-hub.co.uk`

Step 1 — Username (CONFIRMED selectors)

- Form: `#localAccountForm`
- Username/email input: `#signInName` (placeholder "Enter username", aria-label "Username/Email")
- Submit: `#next` (text "Continue")
- Playwright:

```
await page.fill('#signInName', USERNAME);
await page.click('#next');
```

Step 2 — Password (EXPECTED — confirm on real login)

Standard B2C self-asserted layout after Continue:

- Password input: likely `#password`
- Submit: likely `#next` (or `#continue`)
- "Any difficulties logging in?" / forgot-password link present on step 1.

```
await page.fill('#password', PASSWORD); // CONFIRM id
await page.click('#next'); // CONFIRM id
```

Step 3 — MFA (EXPECTED — confirm on real login)

Tenant almost certainly enforces MFA (email/SMS/authenticator code). Selectors unknown until observed. **This is the hard blocker for headless automation** — MFA needs a human. Practical options:

- Human logs in once; agents reuse the authenticated browser context (`storageState`) until it expires.
- Or run agents as "assisted" — human present to clear MFA when prompted.

Post-auth

- On success, B2C returns a code to `redirect_uri (response_mode=fragment)`, MSAL exchanges it for tokens, app lands back on `.../employee/manage`. Capture that page's DOM next (needs auth).

Security notes

- No credentials in repo / agent prompts / logs.
- Reuse a human-established session (`storageState`) rather than scripting the password+MFA. Treat the session token as a secret; never commit it.

- Reaching this site at all required trusting the environment's egress

TLS-inspection CA in Chrome — that's a container detail, not part of the real runbook.

Portal Walkthrough Template

agent - swarm/PORTAL_WALKTHROUGH_TEMPLATE.md

Atlas Portal Walkthrough — fill from ONE manual entry

Complete this by doing a single real "Add result" in Atlas (human-led). Every

<CONFIRM> in the runbook/field-map comes from here. **Swarm does not launch until this is filled.**

Access

- Atlas URL (training/add-result page): _____
- Login method (SSO / user+pass / MFA): _____
- (Agents use an already-authenticated session — record HOW a session is

established for the human, not the credentials themselves.)

The "Add result" flow — capture each step

- Path to the form (menu clicks): _____
- **Staff field:** label = _____; type = (autocomplete / dropdown / search);

does it create-new or only select-existing? _____; what a no-match looks

like: _____

- **Course field:** label = _____; full list exported? (Y/N) → save the list

into FIELD_MAPPING.md. How a missing course presents: _____

- **Completed/Date field:** label = _____; accepted format = _____

(test 03/04/2025 — does it read as 3 Apr or 4 Mar?): _____

- **Expiry field:** label = _____; is it manual or auto-derived from course

interval? _____; how is "no expiry" represented? _____

- **Status/result:** derived from dates automatically, or set manually? _____;

if manual, the available options: _____

- **Save button** label/selector: _____; success confirmation looks like:

_____; is there a confirmation/reference id? _____

- **Dedupe view:** where do existing results for a staff+course show? _____

Anti-bot observations

- Any CAPTCHA / rate warnings during manual entry? _____

- Session timeout length:
- Any "are you sure / bulk" guards:

Selectors (if using Playwright/browser automation)

Paste the resolved CSS/ARIA selectors for each field above so agents are deterministic:

- staff:
- course:
- completed:
- expiry:
- save:
- success indicator:

Swarm README

agent-swarm/swarm/README.md

Atlas Swarm — runnable harness

Parallel scripted Playwright workers that enter the queued training records into the Atlas portal, reusing one human-established login session. This is the fast/cheap swarm for ~1,300 near-identical form-fills. (For the alternative LLM-agent-per-record approach, see `../AGENT_RUNBOOK.md` — not recommended at this volume.)

Files

- `config.py` — portal URL, **selectors**, throttle, date format. Fill the

`TODO` selectors from `../PORTAL_WALKTHROUGH_TEMPLATE.md` before a live run.

- `login_bootstrap.py` — run once (headed), log in + MFA, saves `auth_state.json`.
- `run_worker.py` — one shard of the queue; run several in parallel.
- `merge_results.py` — combine worker outputs → `../work_queue_status.csv` + recon.
- `results/`, `screenshots/` — per-run outputs (gitignored).
- `auth_state.json` — your session (gitignored, **secret**).

Setup (on a machine with a display, your own — not the cloud container)

```
pip install playwright
playwright install chromium
```

Step 1 — capture selectors (once)

Do one real "Add result" in Atlas and fill the `TODO`s in `config.py` per

`../PORTAL_WALKTHROUGH_TEMPLATE.md`. Confirm `ADD_RESULT_URL` and `DATE_FORMAT`.

Step 2 — capture a login session (once, re-run when it expires)

```
python login_bootstrap.py      # browser opens; log in fully incl. MFA; press ENTER
```

Produces `auth_state.json`. **MFA is why this is manual** — it can't be headless.

Step 3 — pilot (no writes)

```
python run_worker.py --worker pilot --start 1 --end 10 --dry-run
```

--dry-run walks the shard and checks selectors/queue without touching Atlas.

Then do a real pilot of 10 and verify in Atlas (see ../QA_VERIFICATION.md)

before scaling.

Step 4 — scale (static shards, non-overlapping ranges)

The queue is priority-sorted (OVERDUE first). Split it across N workers and run them concurrently. Example for the first 296 OVERDUE across 5 workers:

```
python run_worker.py --worker A --start 1 --end 60 &
python run_worker.py --worker B --start 61 --end 120 &
python run_worker.py --worker C --start 121 --end 180 &
python run_worker.py --worker D --start 181 --end 240 &
python run_worker.py --worker E --start 241 --end 296 &
wait
```

Keep N small (3–5) to stay human-like and avoid bot-detection / lockout. Each worker throttles 8–15s/record and stops itself after 3 consecutive errors.

Step 5 — reconcile

```
python merge_results.py
```

Writes ../work_queue_status.csv and prints DONE/SKIPPED/HOLD/ERROR/PENDING

counts. Target: `DONE + SKIPPED + HOLD + ERROR = 1,358`, all 7 HOLD = the

uncertain surnames, ERRORS itemised for follow-up.

Safety / guardrails (built in)

- Reuses a human session; **never handles credentials or MFA**, never commits

auth_state.json.

- HOLD-flagged rows (uncertain surnames) are skipped automatically.
- Dedupe: skips a record if Atlas already has the same staff+course+completed

date (prevents duplicates on re-runs).

- Staff must match an existing record and course must exist — otherwise ERROR,

never a guess / never creates new staff.

- Throttle + consecutive-error stop + per-record screenshots for audit.

Before you go live — confirm

- **Authorisation/ToS:** that automated owner-driven entry into Atlas is

acceptable (or treat as assisted entry). See `ORCHESTRATION.md`.

- **Interval policy** for the 168 CAUTION rows (Makaton / Record Keeping / Risk Assessment / Driver Safety) — fix expiries in `../work_queue.csv` first if 3-year is wrong, so you don't redo them.

Windows Quickstart

agent-swarm/swarm/WINDOWS_QUICKSTART.md

Windows Quickstart

Copy-paste, in order. Use **PowerShell** (Start menu → type "PowerShell" → Enter).

A. One-time setup

```
# 1. Get the files: download the repo ZIP from
#   https://github.com/etblues449/Obsidian-Vault- (green "Code" -> Download ZIP)
#   unzip it, then go into the swarm folder:
cd "$HOME\Downloads\Obsidian-Vault--main\Work\Atlas Training Migration\agent-swarm\swarm"

# 2. Install the browser automation (once):
pip install playwright
playwright install chromium
```

(If `python/pip` aren't recognised, install Python from python.org and tick

"Add Python to PATH" during install.)

B. Step 2 — capture the form fields (needs only a browser, ~5 min)

- Log into Atlas, open **Add result** for any one staff member.
- Press **F12** → **Console** tab.
- Open `capture_selectors.js`, copy all, paste into Console, Enter.
- Send the printed table to Claude → it fills `config.py` for you.

(Or fill it yourself using `SELECTOR_WORKSHEET.md`.)

C. Step 3 — log in once

```
python login_bootstrap.py
```

A browser opens → log in with password + MFA → press Enter. Session saved.

D. Step 4 — check ready

```
python preflight.py      # must say READY
```

E. Step 5 — pilot, then run it all (watch live)

```
# 10-record test first:
python run_worker.py --worker pilot --start 1 --end 10
#   -> check those 10 in Atlas look right, then:
```

```
.\run_swarm.ps1 -Start 1 -End 296 -Workers 5      # the 296 overdue first
.\run_swarm.ps1 -Start 297 -End 1358 -Workers 5   # the rest
python merge_results.py                          # final count = 1,358
```

Each worker opens its own window so you can watch records go in. A screenshot is saved per record in `screenshots\`.

If PowerShell blocks the script

Run once, then retry:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Anything stalls

Copy the window's text to Claude — it diagnoses and tells you the next move.

The only steps Claude can't do are B and C (your Atlas login).

Selector Worksheet

agent-swarm/swarm/SELECTOR_WORKSHEET.md

Selector Worksheet — 5 minutes, fill the blanks

Goal: get the 8 values that make `config.py` live. Two ways — the **fast way** (console snippet) or the **manual way** (right-click Inspect). Do whichever.

FAST WAY (recommended)

- Log into Atlas. Open the **Add result** form for any one staff member

(so the staff/course/date fields are on screen).

- Press **F12** → click the **Console** tab.
- Open `capture_selectors.js`, copy the whole file, paste into the Console,

press **Enter**.

- It prints a table. Each row has a **selector** (like `#staffId`) and a

label (like "Staff member") so you can tell which is which.

- Copy the selector for each field into the blanks below.

FILL THESE IN → then paste into `config.py`

#	What to find on the form	Selector you found	<code>config.py</code> variable
1	The page URL of the Add result form		<code>ADD_RESULT_URL</code>
2	Staff name box		<code>SEL_STAFF_INPUT</code>
3	The dropdown option that appears after typing a name		<code>SEL_STAFF_OPTION</code>
4	Course box		<code>SEL_COURSE_INPUT</code>
5	The course dropdown option		<code>SEL_COURSE_OPTION</code>
6	Completed / Date-taken box		<code>SEL_COMPLETED_DATE</code>
7	Expiry / Renewal box		<code>SEL_EXPIRY_DATE</code>
8	Save button		<code>SEL_SAVE</code>
9	Something that appears AFTER a successful save (e.g. a green "Saved" toast)		<code>SEL_SAVE_SUCCESS</code>

Also note:

- Date format** the box accepts (type `03/04/2025` — does it read as 3 Apr or

4 Mar?): → set `DATE_FORMAT` in `config.py`

(`%d/%m/%Y` for DD/MM/YYYY).

Notes on #3 and #5 (the dropdown options)

These two often need a tiny placeholder. If the option shows the name/course as text, use a Playwright text selector with `{name}` / `{course}` in it, e.g.:

- `SEL_STAFF_OPTION = "li:has-text('{name}')"`
- `SEL_COURSE_OPTION = "li:has-text('{course}')"`

The `{name}` / `{course}` get filled in automatically per record. If you're not sure, paste me what the console table showed for those rows and I'll write the exact line.

MANUAL WAY (if the console snippet doesn't work)

For each field: right-click it → **Inspect** → in the highlighted HTML look for

`id="..."`. Use `#thatId` as the selector. No id? Use `name="..."` →

`[name="thatName"]`.

When done

```
cd agent-swarm/swarm
python preflight.py      # should now say READY  (after login_bootstrap.py too)
```

Stuck on any row? Paste me the console table output and I'll fill `config.py` for you.

Citation Onboarding Cover Note

citation-onboarding/COVER_NOTE.md

Citation Onboarding — Historical Training Bulk Load

From: Select Lifestyles Limited (Learning Disability provider)

Re: Bulk-loading historical staff training records into Atlas

Date: 2026-06-10

What this is

A cleaned, verified set of historical training records for bulk entry into Atlas, so existing currency/expiry shows correctly rather than re-entering 1,300+ records by hand. Compiled from our training database **plus** a full audit of 102 paper attendance sheets (which recovered ~6 months of sessions missing from the spreadsheet).

Files

- **Citation_Bulk_Load.xlsx** — sheet "**Bulk Load**" = the records to load;

sheet "**Held - needs HR name check**" = 7 records excluded pending a name check (don't load these yet).

- **Citation_Bulk_Load.csv** — same Bulk Load data, plain CSV.

Columns

Column	Notes
Full Name	Staff member — should match the Atlas staff record
Training Course	Course name (see mapping note below)
Completed Date	DD/MM/YYYY
Expiry Date	DD/MM/YYYY ; blank = one-time course, no expiry
Interval (yrs)	1 or 3; blank = no expiry. The expiry already reflects this.
Status	OVERDUE / EXPIRING / CURRENT / NO EXPIRY (as at 04/06/2026) — informational

Counts (to reconcile against your load)

- **Records to load: 1,351**
- With expiry date: 1,244
- No expiry (one-time): 107
- By status: OVERDUE 296 · EXPIRING 0–30d 16 · EXPIRING 31–60d 7 · CURRENT 925 · NO EXPIRY 107
- Held / excluded (name check): **7** → 1,351 + 7 = 1,358 verified total
- Unique staff: 408 · Unique courses: ~34

Questions for your onboarding team (please confirm)

- ****Can you bulk-load historical results with our own completion AND expiry dates**** (i.e. set arbitrary/back-dated expiry), rather than Atlas

auto-calculating expiry from a course interval? This matters because some of our expiries are back-dated and several courses are "no expiry".

- **What exact file format / template do you need?** We can match it — column order, date format, a staff/course ID instead of name, separate files per course, etc. Tell us and we'll regenerate.

- **Staff matching:** do you match on full name, email, or an Atlas staff ID? If ID/email, send us the staff list and we'll map.

- **Course matching:** do our course names need to match your Atlas course library exactly? If so, please share the course list so we can reconcile (a few of our labels may differ). Any course not in Atlas — can you create it as part of onboarding?

- **One-time / no-expiry courses (107):** confirm these load with a blank expiry and won't be auto-flagged as overdue.

- **Turnaround & priority:** the **296 OVERDUE** are the priority — can these go first if a staged load is easier?

Notes / assumptions to flag

- **Interval assumptions:** Makaton, Record Keeping, Risk Assessment, Driver Safety aren't in the core CQC list — we defaulted them to a **3-year** expiry.

If your standard differs, tell us and we'll adjust before load.

- **Authority for intervals:** Skills for Care Statutory & Mandatory Training Guide (2025); NICE SC1/NG67; CQC Regulation 18.

- **Held 7:** uncertain handwritten surnames on 2026 attendance sheets — withheld to avoid creating duplicate/mis-named staff; we'll supply once HR confirms.

If bulk load isn't possible

If Atlas can't import historical results with arbitrary expiry dates via your team, we'll fall back to assisted manual entry (priority: the 296 OVERDUE first).

Please confirm which path applies.